

SOC ANALYST PORTFOLIO CASE STUDY

Wazuh Active Response: Automated Containment

A tightly scoped, reversible firewall response triggered by a custom Suricata detection

A level-10 alert blocked only the authorized Parrot source for 120 seconds, then automatically removed the exact labeled rule and restored connectivity.

Analyst	Ryan Bryant
Exercise date	August 29, 2026
Environment	Authorized VirtualBox SOC home lab
Classification	Portfolio demonstration - no production systems

AUTHORIZED-LAB SAFETY NOTE

All scanning, authentication attempts, firewall actions, and log collection occurred inside a controlled private network owned and operated by the analyst.

Executive Summary

This lab extended the validated Suricata-Wazuh detection chain into a controlled response workflow. A request from Parrot (10.10.10.5) to /soc-lab-admin-test on LINUX-WEBSERVER01 (10.10.10.4) produced Suricata SID 1000001. Wazuh custom rule 100120 matched the source, destination, and signature at level 10 and triggered a local Python Active Response script. The script inserted one labeled iptables DROP rule, blocked the authorized source, and logged the action. After 120 seconds, Wazuh issued the delete action; the script removed only its exact labeled rule; and both ICMP and HTTP connectivity returned without analyst intervention.

STAKEHOLDER OUTCOME

The exercise demonstrated a complete detect-contain-recover cycle with prevalidated scope, timed rollback, independent evidence, and no residual firewall rule.

Objective and Scope

- **Primary objective:** Demonstrate automated containment that is selective, reversible, observable, and tied to a high-confidence detection.
- **Detection condition:** Wazuh rule 100120 required Suricata SID 1000001, source 10.10.10.5, and destination 10.10.10.4.
- **Response action:** Insert an iptables INPUT DROP rule for the authorized source with label WAZUH-CONTROLLED-AR.
- **Recovery condition:** Remove only the exact labeled rule after a 120-second Wazuh timeout.
- **Safety boundary:** Private lab hosts only, documented rollback command, backups before changes, and no broad subnet block.

Lab Environment

Role	System	Address / Function
Controlled source	Parrot Security	10.10.10.5 - generated the authorized test request
Protected endpoint	LINUX-WEBSERVER01	10.10.10.4 - Suricata, Apache, Wazuh agent, response script
Manager	wazuh-server	10.10.10.3 - rule 100120 and Active Response orchestration
Timeout	Wazuh Active Response	120 seconds - automatic delete action

Control Design

Guardrail	Implementation	Why it matters
Source allowlist	Only 10.10.10.5	Prevents arbitrary alert sources from becoming block targets
Destination check	Only 10.10.10.4	Binds the response to the protected endpoint
Signature check	SID 1000001 on add	Requires the intended high-confidence detection
Exact rule label	WAZUH-CONTROLLED-AR	Allows precise rollback without touching unrelated firewall rules
Safe execution	subprocess argument lists; no shell	Reduces command-injection risk
Timed recovery	120-second delete action	Limits impact if the response fires incorrectly

Pre-Deployment Validation

The custom script was installed as root:wazuh with mode 750 and executed in self-test mode. The self-test confirmed the expected source, destination, signature, iptables availability, and rule label before Wazuh was permitted to call it.

```
socstudent@linux-webserver01:~$ sudo install -o root -g wazuh -m 750 \
/home/socstudent/controlled-firewall-drop.py \
/var/ossec/active-response/bin/controlled-firewall-drop.py

sudo /var/ossec/active-response/bin/controlled-firewall-drop.py --self-test

sudo stat -c '%U:%G %a %n' \
/var/ossec/active-response/bin/controlled-firewall-drop.py
{
  "allowed_source": "10.10.10.5",
  "expected_destination": "10.10.10.4",
  "expected_signature_id": "1000001",
  "iptables_executable": true,
  "iptables_exists": true,
  "rule_comment": "WAZUH-CONTROLLED-AR"
}
root:wazuh 750 /var/ossec/active-response/bin/controlled-firewall-drop.py
socstudent@linux-webserver01:~$ |
```

Figure 1. Script self-test and ownership verification confirmed the response guardrails and executable permissions.

Wazuh logtest decoded the controlled Suricata record and matched custom rule 100120 at level 10 with MITRE T1190 context. Manager configuration validation returned exit code 0 before services were restarted.

```
[wazuh-user@wazuh-server ~]$ sudo /var/ossec/bin/wazuh-logtest <<'EOF' | tee /home/wazuh-user/active-response-rule-logtest.txt
{"timestamp":"2026-08-29T19:10:00.000000+0000","event_type":"alert","src_ip":"10.10.10.5","src_port":49404,"dest_ip":"10.10.10.4","dest_port":80,"proto":"TCP","alert":{"action":"allowed","gid":1,"signature_id":1000001,"rev":1,"signature":"SOC LAB Suspicious Admin Path Request","category":"Attempted Information Leak","severity":2}}
EOF

Starting wazuh-logtest v4.14.7
Type one log per line

**Phase 1: Completed pre-decoding.
  full event: '{"timestamp":"2026-08-29T19:10:00.000000+0000","event_type":"alert","src_ip":"10.10.10.5","src_port":49404,"dest_ip":"10.10.10.4","dest_port":80,"proto":"TCP","alert":{"action":"allowed","gid":1,"signature_id":1000001,"rev":1,"signature":"SOC LAB Suspicious Admin Path Request","category":"Attempted Information Leak","severity":2}}'

**Phase 2: Completed decoding.
  name: 'json'
  alert.action: 'allowed'
  alert.category: 'Attempted Information Leak'
  alert.gid: '1'
  alert.rev: '1'
  alert.severity: '2'
  alert.signature: 'SOC LAB Suspicious Admin Path Request'
  alert.signature_id: '1000001'
  dest_ip: '10.10.10.4'
  dest_port: '80'
  event_type: 'alert'
  proto: 'TCP'
  src_ip: '10.10.10.5'
  src_port: '49404'
  timestamp: '2026-08-29T19:10:00.000000+0000'

**Phase 3: Completed filtering (rules).
  id: '100120'
  level: '10'
  description: 'Controlled Suricata admin-path detection from 10.10.10.5 to 10.10.10.4'
  groups: '['ids', 'suricata', 'active_response_lab']'
  firedtimes: '1'
  mail: 'False'
  mitre.id: '['T1190']'
  mitre.tactic: '['Initial Access']'
  mitre.technique: '['Exploit Public-Facing Application']'
**Alert to be generated.

[wazuh-user@wazuh-server ~]$
```

Figure 2. wazuh-logtest matched rule 100120, level 10, for the controlled source, destination, and signature.

Controlled Response Lifecycle

The validation captured four distinct states: reachable baseline, trigger request, active containment, and restored post-timeout state. During containment, ping showed 100% loss and curl returned 000. After the timeout, ping succeeded and Apache returned HTTP 200 again.

```

=== CONTROLLED ACTIVE RESPONSE LIFECYCLE ===

PHASE 1: PRE-TRIGGER
2026-08-29T19:21:17+00:00
PING 10.10.10.4 (10.10.10.4) 56(84) bytes of data.
64 bytes from 10.10.10.4: icmp_seq=1 ttl=64 time=1.20 ms
64 bytes from 10.10.10.4: icmp_seq=2 ttl=64 time=1.10 ms

--- 10.10.10.4 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 1.096/1.149/1.202/0.053 ms
Baseline HTTP status: 200

PHASE 2: TRIGGER ONCE
2026-08-29T19:21:18+00:00
Trigger HTTP status: 404
Waiting 10 seconds for Wazuh processing...

PHASE 3: DURING CONTAINMENT
2026-08-29T19:21:28+00:00
PING 10.10.10.4 (10.10.10.4) 56(84) bytes of data.

--- 10.10.10.4 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1030ms

curl: (28) Connection timed out after 3002 milliseconds
Contained HTTP status: 000

Waiting for the 120-second automatic timeout...
Approximately 120 seconds remaining...
Approximately 90 seconds remaining...
Approximately 60 seconds remaining...
Approximately 30 seconds remaining...

PHASE 4: POST-TIMEOUT RESTORATION
2026-08-29T19:23:43+00:00
PING 10.10.10.4 (10.10.10.4) 56(84) bytes of data.
64 bytes from 10.10.10.4: icmp_seq=1 ttl=64 time=0.937 ms
64 bytes from 10.10.10.4: icmp_seq=2 ttl=64 time=1.20 ms

--- 10.10.10.4 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.937/1.067/1.197/0.130 ms
Restored HTTP status: 200
[user@parrot]~$

```

Figure 3. End-to-end lifecycle evidence shows baseline HTTP 200, trigger 404, contained HTTP 000 with packet loss, and restored HTTP 200.

Manager and Endpoint Evidence

The manager alert confirmed rule 100120, level 10, agent 001, source 10.10.10.5, destination 10.10.10.4, and signature 1000001. This record connected the network detection to the response decision.

```
PASS: rule 100120 alert found
matching_alerts: 1
timestamp: 2026-08-29T19:21:19.995+0000
rule_id: 100120
rule_level: 10
description: Controlled Suricata admin-path detection from 10.10.10.5 to 10.10.10.4
agent_id: 001
agent_name: LINUX-WEBSEVER01
source_ip: 10.10.10.5
destination_ip: 10.10.10.4
signature_id: 1000001
signature: SOC LAB Suspicious Admin Path Request
[wazuh-user@wazuh-server ~]$ |
```

Figure 4. Wazuh manager evidence confirms the exact alert that authorized containment.

The endpoint response log recorded one successful add action at 19:21:20.031427+00:00 and one successful delete action at 19:23:20.191461+00:00, an observed duration of approximately 120 seconds. A final iptables check confirmed no containment rule remained.

```
=== ACTIVE RESPONSE POST-TEST VERIFICATION ===
2026-08-29T19:25:55+00:00
PASS: containment rule was automatically removed

=== CONTROLLED RESPONSE LOG RECORDS ===
{"action": "add", "detail": "blocked 10.10.10.5", "script": "controlled-firewall-drop", "status": "success", "timestamp": "2026-08-29T19:21:20.031427+00:00"}
{"action": "delete", "detail": "restored 10.10.10.5", "script": "controlled-firewall-drop", "status": "success", "timestamp": "2026-08-29T19:23:20.191461+00:00"}
socstudent@linux-webserver01:~$ |
```

Figure 5. Active Response logs record the successful add and delete operations, and the post-test confirms automatic removal.

Verified Results

Phase	ICMP	HTTP	Conclusion
Pre-trigger	Successful	200	Baseline connectivity available
Trigger	Successful	404	Lab URI generated the intended detection
Containment	100% loss	000	Only the controlled source was blocked
Post-timeout	Successful	200	Automatic rollback restored connectivity

Control	Verified result
Wazuh alert	Rule 100120, level 10, expected agent/source/destination/signature
Response log	One add and one delete, both status success
Residual rule	Absent after timeout
Evidence integrity	All 18 final files passed SHA-256 verification
Archive integrity	Final archive checksum verified OK

Findings and Limitations

Confirmed findings

- The custom Wazuh rule matched only the intended controlled Suricata record.
- The local response script enforced its source, destination, signature, and rule-label guardrails.
- Containment blocked the authorized source and did not require a service shutdown.
- Wazuh automatically initiated rollback after approximately 120 seconds, and no labeled rule remained.

What was not proven

- The lab did not test distributed endpoints, repeated offenders, IPv6, alternative firewall backends, or manager failover.
- One controlled source and signature do not establish production false-positive or business-impact rates.
- Automated blocking can affect legitimate users; production use requires ownership, escalation, and exception processes.

Recommendations

Priority	Recommendation	Rationale
High	Use allowlists, exact-match conditions, labeled rules, and a short default timeout for every automated block.	Limits blast radius and makes rollback deterministic.
High	Alert on add/delete failures and regularly test that timeout recovery still works.	A failed delete can turn a temporary response into an outage.
High	Require change review, backups, self-tests, and manager validation before enabling a response.	Reduces configuration and execution risk.
Medium	Track response metrics: trigger count, successful adds/deletes, duration, and recurrence.	Supports tuning, accountability, and operational reporting.

Employer-Facing Project Summary

This project demonstrates response engineering across detection, orchestration, endpoint execution, and recovery. The work emphasizes scoped authorization, deterministic rollback, predeployment validation, multi-source evidence, and clear limits on what the lab proves.

Resume Bullet

PORTFOLIO-READY BULLET

Engineered a guarded Wazuh Active Response workflow that converted custom rule 100120 into a source-specific iptables block, verified 120-second automated rollback, and preserved 18 SHA-256-validated evidence artifacts.

Interview Talking Points

- Why source, destination, and signature checks were all required before the firewall action.
- How the exact iptables comment made deletion safe and deterministic.
- Which independent artifacts proved detection, action, containment, and recovery.

Skills Demonstrated

Wazuh Active Response | Suricata | Python | iptables | Automated containment | Rollback | Rule validation | Evidence integrity

AI and Source Transparency

The analyst personally configured the virtual machines, executed the commands, validated the alerts, interpreted the results, captured the screenshots, and preserved the evidence. Generative AI assisted with command sequencing, troubleshooting suggestions, and documentation structure. Technical claims were checked against direct lab evidence and the official references listed below.

References

- Wazuh documentation - Configure Active Response. <https://documentation.wazuh.com/current/user-manual/capabilities/active-response/how-to-configure.html>
- Wazuh documentation - Custom Active Response scripts. <https://documentation.wazuh.com/current/user-manual/capabilities/active-response/custom-active-response-scripts.html>
- Wazuh documentation - active-response configuration. <https://documentation.wazuh.com/current/user-manual/reference/ossec-conf/active-response.html>